



PROJECT GYM

Desenvolvimento Web II

Grupo inf25dw2g10:

Felipe Castilho	a047152
Juliana Moreira	a047188
Marta Vieira	a046756



Introdução

Este projeto constitui o desenvolvimento de **frontend** em **React** para uma plataforma de **gestão** de um **ginásio** que permite que **admins**, **treinadores** e **clientes** consultem e gerem recursos consoante a sua role.

Os recursos disponíveis passam por: criar e gerir **planos** de treino, definir os **exercícios** incluídos em cada plano, registar as **sessões** de treino realizadas, **avaliações** e **metas**.



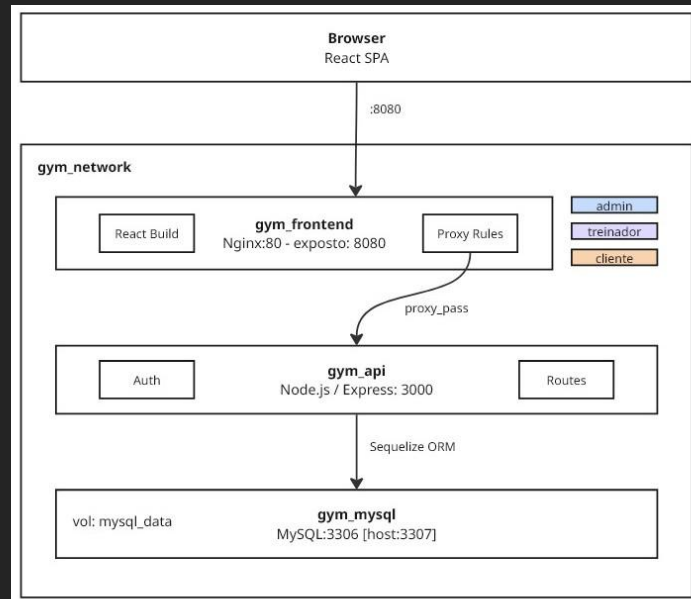
Arquitetura

Apenas o container ***gym_frontend*** tem uma porta exposta ao exterior (8080) os outros dois são internos, apenas acessíveis dentro da rede docker.

Este container é um ***Nginx*** que:

- Serve os ficheiros estáticos do *React* para o browser
- Encaminha pedidos internamente para ***gym_api:3000***

O ***gym_api*** recebe esses pedidos, verifica autenticação, e comunica com a base de dados via ***Sequelize***.



Autenticação

O utilizador entra por **Basic Auth**.

- API devolve **apiKey** e a **role** e *AuthService* guarda-os no *localStorage*.
- *Axios* envia a **apiKey** no header em todos os pedidos.
- *RoleRoute* verifica a **role** e redireciona para o dashboard certo.

O utilizador entra por **OAuth**.

- O provider chama o *callback* na API que gera a **apiKey** e redireciona para o frontend com a **apiKey, role e userId** como **query params**.
- *OAuthCallbackPage* guarda esses parâmetros no *localStorage*

AUTHENTICATION

PROJECT/GYM

UTILIZADOR OU EMAIL

PASSWORD

ENTRAR

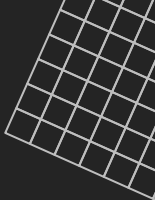
OU ENTRA COM

 GITHUB

 GOOGLE

OAuth redireciona para o provider e regressa ao dashboard da tua role.

OAuth 2.0 · BASIC AUTH



Organização do Código

O código está organizado em `src/` por responsabilidades:

- **pages/** separa as páginas por dashboard (**trainer/**, **client/**, **admin/**).
- **components/common/** concentra componentes reutilizáveis como filtros, modais e layouts.
- **api/** e **auth/** tratam da comunicação com a API e da sessão.

O admin reutiliza as páginas do treinador uma vez que a mesma UI serve para ambos.





Padrões React

As páginas com lógica (login, dashboards e CRUD) são **class components**.

O **state** vive no **construtor** e é atualizado com **setState** para **listagens, filtros, modais e formulários controlados**.

- Só após validação **client-side** é que fazemos POST/PUT à API.

Nos dashboards usámos **lifting state up**: o pai guarda a **tab ativa, plano selecionado** e **filtros**, passando-os como **props** aos filhos.

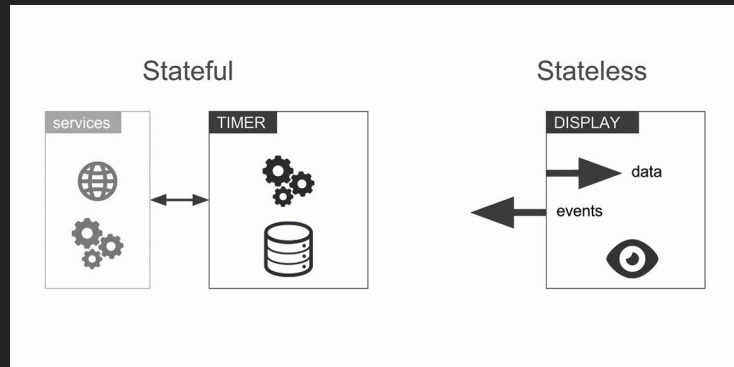
- Ao seleccionar um cliente somos direcionados para a página de planos com o **filtro já aplicado**.

Padrões React

No lifecycle, **`componentDidMount`** carrega dados da API e **`componentDidUpdate`** reage a mudanças de propriedades/props do pai.

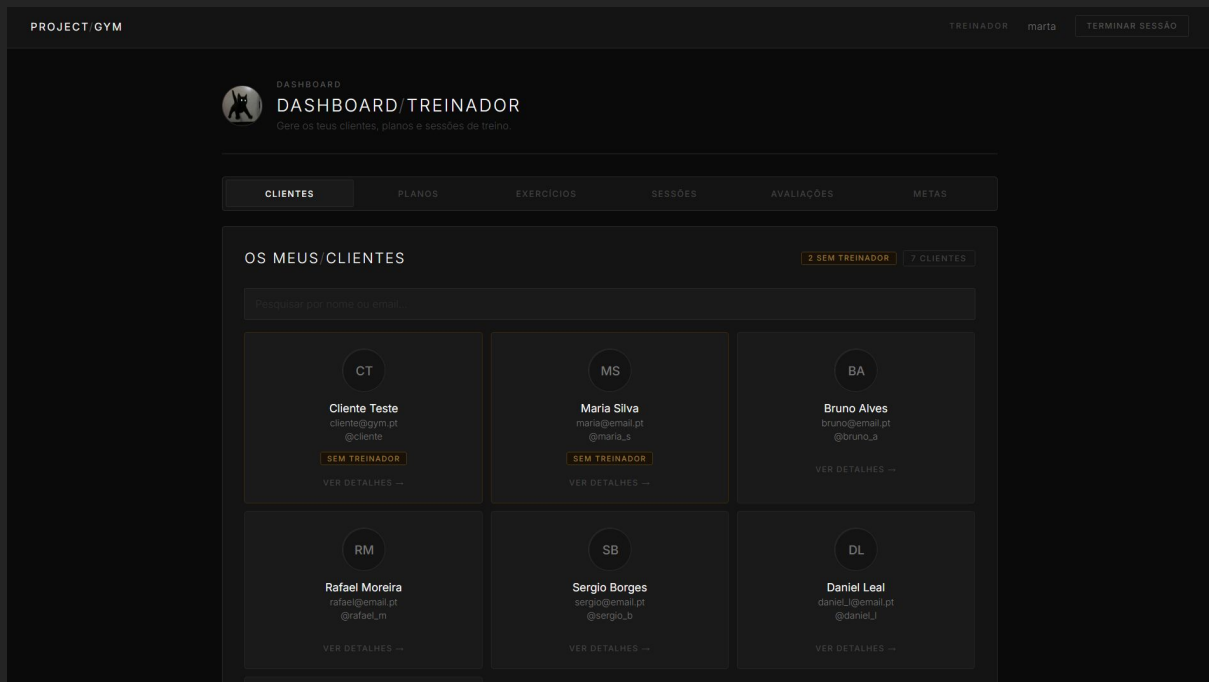
Para interfaces repetidas (**`FilterSelect`**, **`Navbar..`**) usamos *functional* **`stateless`** components:

- O pai é responsável pela lógica e o filho **só apresenta e emite callbacks**.





Dashboard Treinador



Dashboard Admin

PROJECT GYM

ADMINAdmin SistemaTERMINAR SESSÃO

AS

ADMINISTRAÇÃO

PAINEL / ADMINISTRAÇÃO

Gestão total de utilizadores, planos, exercícios, sessões e avaliações.

UTILIZADORES

PLANOS

EXERCÍCIOS

SESSÕES

AVALIACÕES

METAS

Procurar por nome, apelido ou email

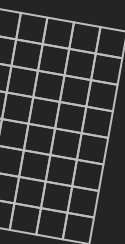
Roles

ATUALIZAR

63 DE 63 UTILIZADORES

#	UTILIZADOR	USERNAME	ROLE ATUAL	ALTERAR ROLE	AÇÕES
48	L Lara Batista lara_batista@pgm.pt	@lara_b	CLIENTE	cliente	APAGAR
33	M Marta Correia marta_correia@pgm.pt	@marta_co	CLIENTE	cliente	APAGAR
34	N Nuno Cardoso nuno_cardoso@pgm.pt	@nuno_ca	CLIENTE	cliente	APAGAR
35	R Rita Henriques rita_henriques@pgm.pt	@rita_h	CLIENTE	cliente	APAGAR
36	S Sergio Borges sergio_borges@pgm.pt	@sergio_b	CLIENTE	cliente	APAGAR
37	M Marta Vieira marta_vieira07@gmail.com	@marta_vieira	CLIENTE	cliente	APAGAR
38	M marta AC46756@umaia.pt	@marta19	TREINADOR	treinador	APAGAR
39	L Luisa Nogueira luisa.nogueira@gym.pt	@treinador_luisa	TREINADOR	treinador	APAGAR
	T Tomás Mota tomás.mota@pgm.pt	@tomas_mota	CLIENTE	cliente	APAGAR

Conclusão



Desenvolvemos uma aplicação Web Client em React para a uma *API REST Gym API* anteriormente desenvolvida com uma **camada de autenticação e autorização** para a gestão de **recursos** consoante o **perfil** do utilizador.

Em suma, o trabalho mostra a integração entre **frontend** e **backend** num cenário de **gestão de ginásio**, com separação clara de responsabilidades entre interface, autenticação e API.

